

一、整合概述

CSG（电鸿新增特性集成）模块包含三个子组件，提供电鸿系统的设备运维、MQTT 通信测试和组件一致性验证功能。本文档记录了 CSG 模块在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- CSG 模块版本：4.0
- 许可证：MIT
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux (ipcamera_hispark_hi3519dv500_linux)
- 目标架构：ARM64 (aarch64)
- 适配模式：模式 D（无参考仓库适配，参考 taurus 平台文档）
- 编译日期：2026-05-18

子组件列表：

组件名称	功能描述	编译产物
hawk_eye_system	鹰眼系统监控组件，提供设备信息采集、网络监控、登录信息管理、SIM/4G 信息、电源状态等功能	power_message、get_login_info、EagleEyeDeamon、sim_message、devinfo、net_message、mytest、预置工具
mqtt_test	MQTT 客户端测试组件，基于华为 IoT Agent 实现 MQTT 协议通信测试	MQTT_Demo、mqtt_config.json
powerbindservice	组件一致性测试工具，基于 CivetWeb 的 HTTP 服务，验证关键组件行为	powerbindservice

适配系统类型：mini、small、standard

二、整合步骤

1. 注册 csg 子系统

文件： build/subsystem_config.json

在文件末尾 } 前追加 csg 子系统注册：

修改前（末尾）：

```
"fuzztest": {
  "path": "test/fuzztest",
  "name": "fuzztest"
}
}
```

修改后（末尾）：

```
"fuzztest": {
  "path": "test/fuzztest",
  "name": "fuzztest"
},
"csg": {
  "path": "base/csg",
  "name": "csg"
}
}
```

修改原因： OpenHarmony 构建系统通过 `subsystem_config.json` 查找子系统路径。csg 子系统位于 `base/csg` 目录，必须在全局配置中注册，否则 `config.json` 中引用 csg 组件时构建系统无法定位。

2. 产品配置添加 csg 组件

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/config.json`

在 `subsystems` 数组末尾（`"communication"` 之后）追加 csg 子系统配置：

追加内容：

```
{
  "subsystem": "csg",
  "components": [
    { "component": "hawk_eye_system", "features": [] },
    { "component": "mqtt_test", "features": [] },
    { "component": "powerbindservice", "features": [] }
  ]
}
```

3. 定义 board_device_type 变量

文件： `device/board/hisilicon/hispark_hi3519dv500/device.gni`

在 `product_config_path` 之后追加 board 设备类型声明：

修改前：

```
product_config_path = "//vendor/hisilicon/${product_name}"
board_camera_path = "//device/board/hisilicon/hispark_hi3519dv500/camera"
```

修改后：

```
product_config_path = "//vendor/hisilicon/${product_name}"
board_device_type = "hi3519dv500"
board_camera_path = "//device/board/hisilicon/hispark_hi3519dv500/camera"
```

修改原因： `board_device_type` 被以下编译目标使用：

- `base/csg/hawk_eye_system/tools/small/BUILD.gn`：根据 board 类型选择 hi3519dv500 对应的预置工

具 (busybox、ethtool、tcpdump、libpcap)

- `base/csg/hawk_eye_system/otp-server/BUILD.gn`: 仅当 `board_device_type == "hi3519dv500"` 时编译 OTP 服务

4. 修复 powerbindservice 编译选项

文件: `base/csg/powerbindservice/BUILD.gn`

在 lite (small 系统) 编译路径中, 移除 `-D_UAPI_LINUX_SOCKET_H` 编译选项:

修改前 (第 25-29 行) :

```
cflags = [  
    "-Wno-int-conversion",  
    "-D_UAPI_LINUX_SOCKET_H",  
    "-D__kernel_sa_family_t=sa_family_t"  
]
```

修改后:

```
cflags = [  
    "-Wno-int-conversion",  
    "-D__kernel_sa_family_t=sa_family_t"  
]
```

修改原因: `-D_UAPI_LINUX_SOCKET_H` 宏阻止 `linux/socket.h` 被包含。在 ARM64 + musl libc 环境下, `sys/socket.h` 中 `sockaddr_storage` 被 `#ifndef __MUSL__` 保护 (不定义), 必须允许 `linux/socket.h` 正常包含以提供该类型。详见问题 1。

5. 修复 mqtt_test 编译选项

文件: `base/csg/mqtt/mqtt_test/BUILD.gn`

在 lite 编译路径中, 移除 `-D_UAPI_LINUX_SOCKET_H`:

修改前 (第 24-28 行) :

```
cflags_c = [  
    "-DOPENSSL_ARM_PLATFORM",  
    "-D_UAPI_LINUX_SOCKET_H",  
    "-D__kernel_sa_family_t=sa_family_t"  
]
```

修改后:

```
cflags_c = [  
    "-DOPENSSL_ARM_PLATFORM",  
    "-D__kernel_sa_family_t=sa_family_t"  
]
```

修改原因: 与 powerbindservice 相同的 `sockaddr_storage` 类型缺失问题。

三、问题解决记录

问题 1: powerbindservice / mqtt_test 编译失败 — sockaddr_storage 类型未定义

错误信息:

```
FAILED: obj/base/csg/powerbindservice/3rd/civetweb/powerbindservice.civetweb.o
sysroot/usr/include/netinet/in.h:281:26: error: field 'gr_group' has incomplete type
sysroot/usr/include/netinet/in.h:286:26: error: field 'gsr_group' has incomplete type
sysroot/usr/include/netinet/in.h:295:26: error: array type has incomplete element type
'struct sockaddr_storage'
```

mqtt_test 编译时同样出现 `sockaddr_storage` 相关错误。

根因分析:

musl libc sysroot 中 `sockaddr_storage` 的定义链断裂:

1. `netinet/in.h` 包含了 `sys/socket.h`, 并使用 `struct sockaddr_storage`
2. musl 的 `sys/socket.h` 中 `sockaddr_storage` 被 `#ifndef __MUSL__` 保护, 在 musl 编译环境下 **不定义**:

```
#ifndef __MUSL__
struct sockaddr_storage {
    sa_family_t ss_family;
    char __ss_padding[128-sizeof(long)-sizeof(sa_family_t)];
    unsigned long __ss_align;
};
#endif
```

3. `linux/socket.h` 中有 `sockaddr_storage` 的定义, 但 `-D_UAPI_LINUX_SOCKET_H` 宏阻止了该头文件被包含
4. CivetWeb / MQTT Agent 源码间接引用了 `netinet/in.h`, 导致编译失败

解决方案: 移除 `-D_UAPI_LINUX_SOCKET_H`, 允许 `linux/socket.h` 正常包含以提供 `sockaddr_storage` 定义。保留 `-D__kernel_sa_family_t=sa_family_t` 确保类型兼容。

关联组件: 这是 hi3519dv500 (ARM64/musl) 平台上的系统性问题, 此前在 BusyBox、iptables、libpcap、ph_mqtt 四个组件上遇到并采用相同方案解决。CSG 是第 5/6 个遇到此问题的组件。

问题 2: otp_server 未编译

现象: hawk_eye_system 的 7 个产物均成功编译, 但 `otp_server` 未生成。

原因: `otp-server/BUILD.gn` 通过 `ldflags` 链接海思 OTP SDK 库 `libss_mpi_otp`, 该库需要从海思 SDK 单独获取。当前编译环境中未提供此库。

当前状态: OTP 功能暂不可用, 不影响其他组件的正常编译和使用。如需 OTP 功能, 需将海思 OTP SDK 库放置到正确路径后重新编译。

四、编译验证

编译环境

```
# 进入编译环境，确保 hb 工具可用
cd <源码根目录>
```

编译命令

1. 设置编译目标

```
rm -rf out
hb set --vendor hisilicon -p ipcamera_hispark_hi3519dv500_linux
```

2. 单独编译 hawk_eye_system

```
hb build -T hawk_eye_system
```

编译结果：

```
[OHOS INFO] ipcamera_hispark_hi3519dv500_linux build success
[OHOS INFO] Cost time: 0:02:15
```

3. 单独编译 mqtt_test

```
rm -rf out
hb set --vendor hisilicon -p ipcamera_hispark_hi3519dv500_linux
hb build -T mqtt_test
```

编译结果：

```
[OHOS INFO] ipcamera_hispark_hi3519dv500_linux build success
[OHOS INFO] Cost time: 0:02:18
```

4. 单独编译 powerbindservice

```
rm -rf out
hb set --vendor hisilicon -p ipcamera_hispark_hi3519dv500_linux
hb build -T powerbindservice
```

编译结果：

```
[OHOS INFO] ipcamera_hispark_hi3519dv500_linux build success
[OHOS INFO] Cost time: 0:02:21
```

生成文件

```
cd out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux
```

hawk_eye_system 产物

```
$ ls -lh bin/power_message bin/get_login_info bin/EagleEyeDeamon bin/sim_message bin/devinfo bin/net_message bin/mytest
-rwxrwxr-x 1 chenzhy223 chenzhy223 14K bin/devinfo
-rwxrwxr-x 1 chenzhy223 chenzhy223 9.8K bin/EagleEyeDeamon
-rwxrwxr-x 1 chenzhy223 chenzhy223 14K bin/get_login_info
-rwxrwxr-x 1 chenzhy223 chenzhy223 9.8K bin/mytest
-rwxrwxr-x 1 chenzhy223 chenzhy223 14K bin/net_message
-rwxrwxr-x 1 chenzhy223 chenzhy223 9.8K bin/power_message
-rwxrwxr-x 1 chenzhy223 chenzhy223 14K bin/sim_message
```

mqtt_test 产物

```
$ ls -lh bin/MQTT_Demo etc/mqtt_config.json
-rwxrwxr-x 1 chenzhy223 chenzhy223 42K bin/MQTT_Demo
-rw-r--r-- 1 chenzhy223 chenzhy223 195 etc/mqtt_config.json
```

powerbindservice 产物

```
$ ls -lh bin/powerbindservice
-rwxrwxr-x 1 chenzhy223 chenzhy223 150K bin/powerbindservice
```

文件格式验证

```
$ file bin/power_message bin/MQTT_Demo bin/powerbindservice bin/devinfo
bin/power_message: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
bin/MQTT_Demo: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
bin/powerbindservice: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
bin/devinfo: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
```

五、功能说明

hawk_eye_system — 鹰眼系统监控组件

工具	功能
power_message	电池/电源状态信息采集
get_login_info	SSH 登录审计记录

工具	功能
sim_message	SIM/4G 模块信息查询
EagleEyeDeamon	鹰眼看门狗守护进程
devinfo	设备信息采集（CPU、内存、存储等）
net_message	网络接口信息采集（IP、MAC、速率等）
mytest	初始化测试工具
otp_server	OTP 安全芯片读取（hi3519dv500 专用，需海思 SDK）

mqtt_test — MQTT 通信测试组件

基于华为 IoT Agent (iota/iotaimp) 实现的 MQTT 协议客户端测试工具，支持：

- MQTT Broker 连接与认证
- 消息订阅/发布测试
- TLS 加密通信（通过 OpenSSL）

配置文件：`etc/mqtt_config.json`（包含 DeviceId、服务器 IP、端口等配置）

powerbindservice — 组件一致性测试工具

基于 CivetWeb 的轻量 HTTP 服务，提供：

- 组件合规性检查
- 关键组件行为验证
- HTTP API 接口供外部调用

六、使用方法

依赖关系

前置依赖组件（需先在 config.json 中注册并编译通过）：

依赖项	子系统	被依赖组件
openssl	thirdparty	mqtt_test
ph_mqtt	thirdparty	mqtt_test
init_lite	startup	全部三个组件
cJSON (lite)	build/lite	hawk_eye_system、mqtt_test

预置工具部署

hawk_eye_system 的 tools/small/BUILD.gn 根据 board_device_type 自动选择 hi3519dv500 对应的预置二进制部署到 usr/bin/ 和 usr/lib/:

工具	安装位置	来源组件
busybox	usr/bin/	BusyBox
ethtool	usr/bin/	Ethtool
tcpdump	usr/bin/	Tcpdump
libpcap.so.1	usr/lib/	Libpcap
btools	usr/bin/	海思工具

七、与其他组件的关系

被依赖关系

CSG 模块是终端用户态工具集，不被其他组件依赖。

依赖链

```
CSG (csg 子系统)
├─ hawk_eye_system
│   ├── cJSON (build/lite/config/component/cJSON)
│   └─ libbegetutil (base/startup/init)
├─ mqtt_test
│   ├── ph_mqtt (third_party/ph_mqtt) → openssl (third_party/openssl)
│   ├── cJSON (build/lite/config/component/cJSON)
│   └─ openssl (third_party/openssl)
└─ powerbindservice
    ├── libbegetutil (base/startup/init)
    ├── pthread (系统库)
    └─ 内置 cJSON + CivetWeb (无需外部依赖)
```

八、注意事项

1. 适配模式判定

CSG 适配属于 模式 D (无参考仓库适配) + cflags 宏修复。

2. 平台差异

本产品 (hispark_hi3519dv500_linux) 与参考平台 hispark_taurus_linux (ARM 32-bit) 的主要差异:

- 架构: ARM64 (aarch64) vs ARM 32-bit
- 工具链: aarch64-v01c01-linux-musl-gcc vs arm-linux-ohosclibc-gcc

- 动态链接器： `/lib/ld-musl-aarch64.so.1` vs `/lib/ld-musl-arm.so.1`
- musl libc 头文件差异：ARM64 musl sysroot 中 `sockaddr_storage` 的定义方式不同

3. `-D_UAPI_LINUX_SOCKET_H` 系统性问题

该宏是 hi3519dv500 (ARM64/musl) 平台上的 **通用问题**，已在以下组件中遇到并采用相同方案（移除该宏）解决：

1. BusyBox — `third_party/busybox/BUILD.gn`
2. iptables — `third_party/iptables/extensions/BUILD.gn`
3. libpcap — `third_party/libpcap/BUILD.gn`
4. ph_mqtt — `third_party/ph_mqtt/BUILD.gn`
5. CSG powerbindservice — `base/csg/powerbindservice/BUILD.gn`
6. CSG mqtt_test — `base/csg/mqtt/mqtt_test/BUILD.gn`

任何从 ARM32 平台移植过来的组件，如果 `BUILD.gn` 含此宏，在 hi3519dv500 上都必须移除。

4. OTP 功能限制

`otp_server` 仅在 hi3519dv500 平台编译（受 `board_device_type` 控制），且依赖海思 OTP SDK 库（`libss_mpi_otp`）。当前环境未提供该库，OTP 功能暂不可用。

5. MQTT 运行时依赖

MQTT_Demo 运行时需要：

- 正确配置 `mqtt_config.json`（MQTT 服务器地址、端口、设备 ID）
- 网络连通性（设备与 MQTT Broker 之间）
- OpenSSL 动态库已部署到设备

6. 构建系统适配

所有 `BUILD.gn` 文件通过 `if (defined(ohos_lite))` 区分 lite/standard 编译路径。hispark_hi3519dv500_linux 产品类型为 "small"，使用 lite 路径。

九、修改文件汇总

#	文件路径	修改类型	修改说明
1	<code>build/subsystem_config.json</code>	新增	末尾追加 csg 子系统注册
2	<code>vendor/hisilicon/hispark_hi3519dv500_linux/config.json</code>	新增	subsystems 末尾追加 csg 三个组件
3	<code>device/board/hisilicon/hispark_hi3519dv500/device.gni</code>	新增	追加 <code>board_device_type = "hi3519dv500"</code>
4	<code>base/csg/powerbindservice/BUILD.gn</code>	修改	lite 路径 cflags 移除 <code>-D_UAPI_LINUX_SOCKET_H</code>
5	<code>base/csg/mqtt/mqtt_test/BUILD.gn</code>	修改	lite 路径 cflags_c 移除 <code>-D_UAPI_LINUX_SOCKET_H</code>

十、参考资源

- CSG 模块说明: `base/csg/README.md`
- 鹰眼组件文档: `base/csg/hawk_eye_system/README.md`
- MQTT 测试组件文档: `base/csg/mqtt/mqtt_test/README.md`
- 组件一致性测试工具文档: `base/csg/powerbindservice/README.md`
- OpenHarmony 构建系统: <https://gitee.com/openharmony/docs/blob/master/zh-cn/device-dev/subsystems/subsys-build-all.md>